PRODUCT / SYSTEM SPECIFICATION

RELEASEBUFF / PRD.02 / V1.1

Find the story worth sharing.

A product requirements document for the product, agent, interface and publishing workflow.

01	02	03	04
CAPTURE	INTERPRET	DRAFT	PUBLISH

DESIGN REFERENCE

Tim's Lab — Intelligent Systems for Modern Businesses

https://timslabs.com/

DOCUMENT CONTROL	VALUE
Version	1.1
Date	2 November 2026
Status	Updated PRD covering Product Intelligence, evidence-backed story discovery, adaptive interviews and publishing architecture
Product stage	Local MVP prototype; newly agreed capabilities require implementation and end-to-end validation
Document purpose	Single source of truth for product design, Google Stitch handoff, architecture and implementation planning

Product principle

Releasebuff must understand the product before it tries to tell its story. It combines source-backed facts, builder-provided perspective and adaptive interviewing to produce distinct, truthful stories. Every factual claim must be traceable, and the builder remains in control of the final wording and publishing action.

READ THIS FIRST

Document map

REF	SECTION
01	Executive summary and product definition
02	Problem, goals and non-goals
03	Audience and product principles
04	User journey and screen inventory
05	Functional requirements by screen
06	Story taxonomy: all 16 categories
07	Evidence sources and optional integrations
08	Agent workflow, ranking and content safety
09	Buffer publishing and OAuth security
10	Technical approach, local-first storage and deployment
11	Design direction: Tim’s Lab reference
12	Accessibility, responsive behavior and states
13	Success metrics and acceptance criteria
14	Release plan, risks and open decisions
A1	PRD v1.1 addendum: Product Intelligence, adaptive interviews and draft quality
15	Reference links

At a glance

ITEM	DECISION
Product	AI product companion that learns product context, discovers evidence-backed stories, interviews builders to fill gaps, drafts in their voice and publishes through Buffer.
Primary starting input	Required product name and description; public GitHub URL or GitHub App connection for selected private repositories is optional but recommended; no website is required.
Primary workspace	One unified Story Inbox with category filters, evidence links, proposed angle, missing-context status and story lifecycle.
Content model	16 story types across four groups; only create discovered candidates when evidence and/or explicitly attributed builder context supports them.
Publishing	Buffer OAuth with Save draft, Schedule and Publish now, using real authorized channels and explicit confirmation.

ITEM	DECISION
Storage	Browser IndexedDB for product materials, evidence, profiles, stories and drafts; localStorage for small preferences; Netlify Blobs for encrypted persistent Buffer authorization state. No traditional relational database.
Later capabilities	Automatic GitHub monitoring, advanced video analysis, direct integrations to third-party documentation/analytics/feedback services and cross-device sync.

One-line product promise

Build the product. Connect your evidence. Tell your story.
Releasebuff learns what a builder is creating, finds meaningful moments in the work, interviews the builder when context is missing, and turns the result into truthful social content that can be reviewed and published through Buffer.

01 / PRODUCT DEFINITION

Executive summary

Releasebuff is an AI product companion for solo builders, indie hackers and small SaaS teams who want to build in public without turning content creation into a second job. It learns about a product from the builder's description and available sources, including public or authorized private GitHub repositories, PRDs, product documents, screenshots, demos, public URLs and builder-provided context. It develops a Product Intelligence Profile, discovers evidence-backed story opportunities, interviews the builder when important context is missing, and turns a reviewed Story Brief into natural, platform-aware content. Builders can optionally shape their opening and closing, then save, schedule or publish through Buffer.

The product's core value is understanding the builder's product well enough to identify the story behind the work. A code change may show what changed, a PRD may explain what was intended, and the builder interview may explain why the decision mattered. Releasebuff combines these different kinds of knowledge while keeping source-supported facts, builder-provided context and unresolved hypotheses distinct.

Product definition

DIMENSION	PRD DECISION
Primary user	A solo builder, indie hacker or small SaaS team, including pre-launch builders with an idea, prototype or private/local codebase.
Core job to be done	"Help me understand what is worth sharing about my product, capture the context behind it, and tell that story naturally with evidence and minimal effort."
Primary output	A reviewed Product Intelligence Profile, curated evidence-backed story opportunities, story-specific interview, Story Brief, strong editable drafts and Buffer actions.
Initial evidence sources	Builder description and answers; public or GitHub-App-authorized private repository data; uploaded/pasted PRDs and notes; accessible public URLs; screenshots/demos and manually supplied evidence.
Primary channels	LinkedIn-first drafting; X supported when compatible with the selected Buffer channel and its capabilities.
Primary deployment direction	Next.js application and server-side Netlify functions/routes; IndexedDB/localStorage for browser-local product data; Netlify Blobs for encrypted persistent integration state; secrets in environment variables.
Design handoff	Google Stitch prototype follows this updated PRD; the local implementation must distinguish implemented features from planned capabilities.

Core product loop

STAGE	WHAT HAPPENS	USER VALUE
01 / Discover	Collect the builder description and any supplied repository, product documents, visuals or public URLs. Support public GitHub URL and GitHub App private-repo installation.	Start with whatever the builder has.
02 / Understand	Create a source-attributed Product Intelligence Profile and let the builder correct it. Identify unknowns without inventing answers.	Build product context before writing.

STAGE	WHAT HAPPENS	USER VALUE
03 / Find	Cluster related artifacts into story candidates, map claims to evidence, and choose promising angles across the 16 categories.	Discover the signal among routine work.
04 / Interview and brief	Ask enough adaptive questions to fill story-specific gaps, then create a Story Brief with audience, angle, sources and claim constraints.	Capture the builder’s perspective.
05 / Draft and publish	Generate, verify and edit post copy; optionally let the builder shape the opening/closing; save, schedule or publish through Buffer after confirmation.	Complete the workflow with the builder in control.
06 / Improve	Save permitted interview answers and draft preferences locally; use feedback to avoid repeats and improve relevance.	Stories get more personal over time.

02 / STRATEGY

Problem, goals and non-goals

Problem statement

Builders have a continuous stream of product work but often struggle to turn that work into consistent public storytelling, including before launch. Commits, PRs and roadmaps are useful signals but rarely explain the full product, the builder's motivation, the intended audience or the lessons learned. Some builders have no live website, no customers and no public repository yet. A generic writing assistant can produce fluent copy but often lacks product understanding, evidence and the builder's actual perspective.

Releasebuff closes the gap between building and communicating by combining product discovery, multi-source evidence collection, story selection, an adaptive builder interview, a structured Story Brief, high-quality drafting, post-draft verification and controlled publishing through Buffer.

Goals

- Help a builder find a small number of genuinely useful story opportunities, not a noisy feed of every repository event.
- Ground factual claims in source artifacts and make those artifacts inspectable from every story card.
- Support all 16 story categories from day one as a taxonomy, while only surfacing stories that have sufficient evidence or context.
- Use an adaptive multi-question interview that continues until enough story-specific information is captured or remaining unknowns can be safely omitted.
- Create an end-to-end workflow from product description and available sources through Product Profile, evidence-backed story discovery, adaptive interview, Story Brief, verified draft, and Buffer draft/schedule/publish actions.
- Keep onboarding lightweight and avoid a traditional relational database; use IndexedDB for local product/evidence/draft data and Netlify Blobs for minimal encrypted persistent Buffer authorization state.
- Use the Tim’s Lab reference for a confident, systems-oriented visual language with clear stages, numbered modules and concrete before/after framing.

Non-goals for the initial release

- Automatic GitHub monitoring, webhooks or continuous background scanning. These are planned later features.

- Broad GitHub permissions, repository write access, and automatic background monitoring. Private repositories are in scope through a GitHub App with user-selected repositories and least-privilege read-only access; monitoring remains later phase.
- Auto-publishing without a user confirmation step.
- Claims that an implementation increased revenue, users, productivity or conversion unless verified data supports the claim.
- Direct integrations with analytics, customer-support tools, feedback systems, roadmap software or documentation platforms in the first release. Manual upload, paste and accessible URL intake for supported material are in scope.
- A requirement to use a relational database for repository URLs, local story status or local draft history.
- Guarantees of virality, follower growth or business outcomes.

Product scope guardrail

The MVP includes GitHub App authorization for selected private repositories; public URL-based scans; manual evidence intake for documents, notes and visuals; Product Intelligence Profile; adaptive story interviews; Story Brief and draft quality checks; and real Buffer OAuth/post actions once implemented and tested. Automatic GitHub monitoring and direct third-party integrations beyond GitHub/Buffer remain later-phase features.

03 / USERS

Audience and product principles

Primary audience

PERSONA	NEED	DESIGN IMPLICATION
Indie hacker / solo founder	Grow an audience while shipping a product, without turning content into a second job.	Fast setup, small curated inbox, short drafts and low friction.
Developer building a tool	Share technical progress, engineering decisions, debugging stories and product demos.	Precise source references, technically literate language, no invented rationale.
Small SaaS team	Communicate releases, learning, customer outcomes and roadmap progress.	Allow multiple story sources over time and make metrics/feedback evidence explicit.
Early product creator	Attract first users and get useful feedback while the product evolves.	Support launch, audience-question and product-showcase categories, but ask for context when needed.

Product principles

PRINCIPLE	EXPECTED BEHAVIOR
Understand before writing	Build a persistent, reviewable Product Intelligence Profile before trying to tell stories about the product.
Evidence before eloquence	Prefer modest, supportable claims over impressive but unverified narratives.
Show the source	Every source-driven opportunity and factual draft claim should point to relevant evidence, with excerpts and dates where available.
Interview until ready	Ask a sequence of adaptive story-specific questions until material gaps are resolved or explicitly marked unknown; do not stop after one question by default.
One inbox, many categories	Keep one unified feed; category selection filters the same feed rather than creating 16 isolated inboxes.
Human approval at the boundary	The agent can prepare content; publish and schedule actions require explicit review and confirmation.
Builder’s voice	Use source facts and interview answers, support optional custom opening/closing, and avoid template-like repetition.
No fabricated activity	Do not invent customer quotes, founder emotions, failed experiments, product availability, user metrics or outcomes.

Primary success outcomes

- Time to first useful story: the time from adding a repo to viewing an evidence-backed opportunity that the builder considers relevant.
- Story usefulness: share of suggested opportunities that are drafted, saved, or marked useful versus dismissed.
- Draft completion: share of created drafts that are edited and saved or sent to Buffer.

- Publishing reliability: successful Buffer draft creation, scheduling and publish-now actions divided by valid requests.
- Trust and factuality: source coverage, correction rate, duplicate story rate, and unsupported-claim incidents.

04 / EXPERIENCE ARCHITECTURE

User journey and screen inventory

Primary journey

STEP	SCREEN / ACTION	EXPECTED RESULT
1	Landing page → “Find your first story”	User understands that Releasebuff helps them build in public before and after launch.
2	Product Setup → product name and plain-language description	Collect minimum product context; website and repository are optional.
3	Add available sources	Paste a public GitHub URL or connect GitHub App for selected private repositories; upload/paste product docs or add public URLs/screenshots.
4	Analyze and review Product Intelligence Profile	Combine source material and builder input; show facts, builder statements and unresolved hypotheses separately.
5	Story Inbox	Manual scan returns curated, deduplicated, evidence-backed story candidates across the 16 categories.
6	Story Details & Interview	Review evidence and proposed angle; conduct adaptive interview until enough story context is captured.
7	Story Brief and Draft Studio	Show main point, audience, angle and sources; optionally personalize opening and closing; generate, verify and edit post.
8	Buffer action	Connect Buffer, select real channel, then save draft, schedule or publish after explicit confirmation.
9	Draft Library	Recover local drafts and approved interview context; continue editing or use Buffer actions.

Seven core screens; Product Profile and sources live in Project Setup

#	SCREEN	PRIMARY JOB	KEY ELEMENTS
01	Landing page	Explain value and start onboarding.	Statement-led hero; product preview; how it works; categories preview; future integrations; CTA.
02	Project Setup / Product Profile	Learn about product and manage evidence.	Product name/description; stage; optional audience; public repo URL or connect GitHub; private-repo selection; optional document upload/paste; public URLs; screenshot/demo intake; source list; profile review and corrections.
03	Story Inbox	Find and triage story opportunities.	Unified feed; categories/status; story cards; supporting evidence coverage; concise “why this matters” angle; source freshness; manual scan/refresh.
04	Story Details & Context	Verify the story and interview the builder.	Source bundle; fact/interpretation split; gaps checklist; dynamic one-question-at-a-time interview; progress/resume/skip; answer attribution; angle selection.

#	SCREEN	PRIMARY JOB	KEY ELEMENTS
05	Story Categories	Explain the 16 content possibilities.	Four groups; compact expandable cards; examples; evidence needs; filter inbox action.
06	Draft Studio	Turn the Story Brief into strong, reviewable content.	Story Brief; narrative angle; platform-aware editor; source drawer; optional opening and closing controls; regenerate/refine; post-draft claim check; Buffer save/schedule/publish.
07	Draft Library	Recover and continue content work.	Saved drafts; status; channel; search/filter; open/edit/copy/continue actions.

Shared navigation

Project Setup should remain accessible from the workspace and own the Product Intelligence Profile and source library. Story Categories is a filter/view, not a new content silo. Story Details & Context owns the adaptive interview. Draft Studio opens from a selected story and returns the user to the same inbox/filter context.

05 / REQUIREMENTS

Functional requirements by screen

01. Landing page

ID	REQUIREMENT	PRIORITY
LP-01	Use the headline “Build the product. Find the story worth sharing.” or an approved equivalent. The value proposition should focus on story discovery, not generic AI writing.	Must
LP-02	Provide primary CTA “Find your first story” and secondary CTA “See how it works”.	Must
LP-03	Show a realistic Story Inbox preview with category labels, source evidence, and the reason the story matters; clearly label any sample/demo data.	Must
LP-04	Explain the flow: describe product → add available evidence → understand product → discover stories → interview → draft/refine → publish through Buffer.	Must
LP-05	Describe GitHub URL scanning and private-repository connection truthfully. Label automatic ongoing monitoring as “Coming soon” until implemented.	Must
LP-06	Show optional PRD/docs, public URLs, screenshots and demos as available intake options. Distinguish manual upload/paste from future direct third-party integrations.	Must
LP-07	Provide a “Connect Buffer” path where relevant, but distinguish Buffer publishing authorization from Releasebuff app identity/authentication.	Must

02. Project Setup

ID	REQUIREMENT	PRIORITY
PS-01	Collect required product name and plain-language description; ask product stage. Target audience is optional initially.	Must
PS-02	Accept public GitHub URL such as https://github.com/owner/repository; normalize trailing slash and optional .git suffix.	Must
PS-03	Allow public URL scan without account authorization. Offer “Connect GitHub” using a GitHub App for selected private repositories.	Must
PS-04	Show GitHub App installation, repository selection, authorization failure, revocation, inaccessible repository and rate-limit states.	Must
PS-05	Allow PRD/document uploads, pasted text, accessible public URLs, screenshots and demo materials where supported; all except product name/description are optional.	Must
PS-06	Build a Product Intelligence Profile that distinguishes source-supported facts, builder-provided statements and unconfirmed hypotheses. Let builder edit/confirm it.	Must
PS-07	Ask follow-up questions only when needed to avoid material assumptions or unlock a useful story. Keep optional profile fields skippable.	Must
PS-08	Let the user replace/add/remove sources without unintentionally deleting unrelated drafts; confirm destructive local-state resets.	Must
PS-09	Display source-fetch status, content type, date collected, access errors and retry/remove options for every supplied source.	Must
PS-10	Provide an explicit “no website/repo yet” continuation path using builder description and contextual interview.	Must

03. Story Inbox

ID	REQUIREMENT	PRIORITY
SI-01	Display one unified feed for all story categories. Do not create 16 separate inboxes.	Must
SI-02	Every card shows title, category, summary, why it matters, proposed angle, evidence coverage/source links, context status and a primary action.	Must
SI-03	Support filters for category and story status; preserve selected filters when opening and returning from details/draft.	Must
SI-04	Offer manual scan/refresh in the initial version. Show scan timestamp and explain when results are based on sample/demo data.	Must
SI-05	Use New, Needs context, Ready to draft, Drafted, Scheduled, Published and Dismissed states. Do not mark published without Buffer confirmation/status evidence.	Must
SI-06	Cluster related repository artifacts and other source items into one coherent candidate; avoid duplicate opportunities across repeated scans.	Must
SI-07	Do not populate a category just to fill space. Empty categories should explain what evidence/context would enable them.	Must
SI-08	Support search and sort by relevance/freshness when a practical volume of stories exists.	Should
SI-09	Clearly separate discovered evidence-backed stories from builder prompts or ideas that do not yet have evidence. Unsupported prompts must not masquerade as discovered stories.	Must
SI-10	Allow the builder to correct a story, provide additional source material, launch the interview or mark it already shared.	Must

04. Story Details & Context

- Show linked evidence artifacts and the relationship between them, including repository artifacts, uploaded/pasted documents, public URLs, visual observations and builder-provided context.
- Separate verified facts from suggested interpretation. Show missing context explicitly instead of blending assumptions into the narrative.
- Allow the user to select/revise the angle, start or resume the adaptive story interview, pause/skip a question, provide more evidence, mark the story irrelevant, or move to the Story Brief/Draft Studio.
- Ask for confirmation or supporting evidence before using metrics, quotes, customer results, launch dates, roadmap commitments, personal emotions or causal claims not supported by sources. Do not imply private source material proves public outcomes.

05. Story Categories

- Show the 16 categories under four groups; each card includes a one-sentence description, short example and required evidence/context.
- Selecting a category filters the existing Story Inbox and visibly indicates the active filter.
- State that a category is always available but may have no current story opportunities.
- Distinguish source-driven categories from those needing builder, analytics or customer-feedback inputs.

05 / REQUIREMENTS

Functional requirements by screen (continued)

06. Draft Studio

ID	REQUIREMENT	PRIORITY
DS-01	Create drafts from a structured Story Brief containing product context, event, source bundle, audience, strongest angle, interview answers and claim constraints—not directly from a raw topic or commit.	Must
DS-02	Preserve source references and claim-to-evidence mapping separately from post body; include source links in post only when appropriate or selected.	Must
DS-03	Allow editing, alternate angles, tone/length refinement, and localized regeneration without losing source facts or interview context.	Must
DS-04	Provide three distinct Buffer actions: Save draft to Buffer, Schedule, Publish now.	Must
DS-05	Before a live action, display destination channel and exact action; require explicit final confirmation for Publish now and Schedule.	Must
DS-06	Scheduling requires selected channel, date, time, timezone, delivery mode where supported, and a visible summary of the local publishing time.	Must
DS-07	Convert selected local date/time to an ISO UTC timestamp before calling Buffer; reject past/invalid dates and handle daylight-saving ambiguity for supported zones.	Must
DS-08	Only show success after Buffer confirms success and returns a post result. Preserve text after API failure and show safe retry/draft fallback actions.	Must
DS-09	Support local save/copy as fallback when Buffer is disconnected or unavailable.	Must
DS-10	Channel metadata, post format and scheduling capabilities must match the selected channel; do not show unsupported actions as available.	Must
DS-11	Offer optional story-specific opening and closing preferences: choose a suitable opening/ending, select a style, or supply exact wording. Allow each preference to be skipped independently.	Must
DS-12	If the builder supplies custom opening/closing text, preserve it faithfully; do not rewrite it unnecessarily. Store per-story choices separately from general style preferences.	Must
DS-13	Run a post-draft factuality check; flag or remove unsupported claims introduced during generation and link factual claims back to evidence where practical.	Must
DS-14	Vary structure and openings naturally; do not force a question, inspirational lesson, CTA or hashtags into every post. Avoid repetitive generic AI phrasing.	Must

07. Draft Library

- List locally available drafts with title/first-line preview, category, platform, updated date and status.
- Allow opening, editing, copying, deleting and continuing to Buffer actions.
- Keep local-only drafts distinguishable from drafts confirmed by Buffer.
- Provide empty, loading, deleted, and stale/missing Buffer post states.
- Explain that browser storage does not synchronize across devices and can be cleared by the browser.

Cross-screen states

AREA	REQUIRED STATES
Repository setup	Empty, valid entry, invalid format, repository not found/private, loading, scan partial, scan complete, rate-limited, retryable server error.
Story Inbox	First-time empty, loading, populated, no matches, category empty, error retrieving sources, no newly discovered stories.
Story details	Evidence available, conflicting artifacts, missing context question, inaccessible source, user dismisses or confirms angle.
Buffer connection	Not connected, redirecting, consent denied, callback success, callback failure, expired/revoked connection, reconnect required.
Publishing	Editing, confirming, request in progress, success, recoverable error, validation error, unknown outcome/timeout requiring status reconciliation.
Storage	Saved locally, save failure/quota exceeded, user clears local data, repository changed.
Product Profile and source intake	No source supplied, source fetched, unsupported type, upload in progress, extraction failed, profile ready for review, conflicting facts, assumption awaiting confirmation, user corrected/confirmed profile.
GitHub App connection	Not connected, installation redirect, selected repos, insufficient permissions, installation not found, revoked access, private repo unavailable, token creation failure.
Builder interview	Not needed, in progress, paused, resumed, skipped question, answer saved, follow-up required, readiness check passed, still needs context.
Netlify Blobs integration state	Token record absent, stored, refreshed, refresh in progress, conflicting refresh attempt, expired/revoked authorization, storage error.

06 / CONTENT MODEL

Story taxonomy: all 16 categories

The taxonomy describes the kinds of stories a builder may want to tell. A category is not a promise that a post is currently available. Each candidate must be grounded in evidence and may require context from the builder.

CATEGORY	GROUP	POTENTIAL EVIDENCE	AGENT RULE
1. New feature shipped	Product progress	Merged PRs, release notes, feature commits, docs.	Can be proposed when shipped/released status is clear; do not equate an open PR with a shipped feature.
2. Product improvement	Product progress	Follow-up PRs, tests, related code changes, before/after screenshots.	Group refinement activity; avoid claims like “faster” unless measured or explicitly evidenced.
3. Bug fixed / problem solved	Product progress	Issue → fix PR/commit → regression test.	Strong candidate when the problem and fix are traceable.
4. Weekly build recap	Product progress	Recent related commits, merged PRs, releases and docs.	Cluster the week into a few themes; omit noisy maintenance.
5. Technical decision / trade-off	Learning and craft	PRD, ADR, PR discussion, code/config change and builder explanation.	Detect a decision; ask for rationale when it is not written down.
6. Lesson learned	Learning and craft	Debugging history, tests, docs and builder reflection.	Suggest a lesson only as a hypothesis until the builder validates it.

CATEGORY	GROUP	POTENTIAL EVIDENCE	AGENT RULE
7. Failed experiment / changed direction	Learning and craft	Reverts, alternatives, branches, experiment notes and builder input.	Never infer “failure” solely from a revert; ask what was learned.
8. Behind the scenes	Learning and craft	Architecture docs, detailed implementation, test strategy, code review.	Show specific engineering detail without implying user impact.
9. Product showcase / demo	Product marketing	Website, feature docs, screenshots, demo video, workflow.	Need accurate explanation of current product behavior; visuals may require the builder.
10. Problem and solution	Product marketing	PRD, positioning copy, target audience info, customer research.	Do not claim widespread demand without market/customer evidence.
11. Product hype / launch promotion	Product marketing	Release notes, launch plan, confirmed date, live landing page.	Confirm launch timing, availability and CTA before drafting.
12. What’s next / roadmap	Product marketing	Roadmap doc, GitHub Projects, planned issues and builder confirmation.	Treat unmerged issue/roadmap items as plans, not promises.
13. Ask the audience	Audience and growth	Open product choice, unresolved trade-off, roadmap question, builder goal.	Ask the builder what feedback they want and whom they want to hear from.
14. Traction and milestones	Audience and growth	Product analytics, billing/signup data, verified metrics supplied by builder.	Requires real numbers and definitions; GitHub stars alone are not customer traction.
15. Customer feedback and learning	Audience and growth	Customer interviews, surveys, support tickets, reviews, feedback notes.	Needs permission and safe handling; remove personal/confidential details.
16. Founder journey / personal reflection	Audience and growth	Builder-written notes, reflection prompts, voice/profile context.	Never fabricate emotions, motivations, doubts or personal experiences.

07 / DATA ARCHITECTURE

Evidence sources and optional integrations

MVP evidence sources

SOURCE	WHAT TO CAPTURE	PRIMARY USE	SCOPE
Builder description and answers	Product name, plain-language description, stage, optional audience and goals, motivations and personal context when supplied.	Initial profile; rationale and voice context.	MVP
GitHub repositories (public or selected private)	Repo metadata, README, default branch and dates; use GitHub App installation token for selected private repositories.	Product understanding, development evidence and story candidates.	MVP
Commits and selected diffs	Messages, dates, changed paths; retrieve relevant diff detail for candidate stories only.	Progress, implementation decisions and technical detail.	MVP
Pull requests and issues	Titles, bodies, dates/status, linked artifacts and relevant discussion where permissioned.	Feature journeys, problem/solution and bug-fix stories.	MVP
Releases and changelogs	Version, release notes, tag/date and documented feature state.	Shipped features, release stories and recaps.	MVP
Uploaded/pasted product docs and builder notes	PRDs, specifications, roadmaps, ADRs, meeting/experiment notes, changelogs and relevant text or markdown.	Purpose, audience, intention, technical rationale and future direction.	MVP
Accessible public URLs	Product pages, public docs, blog posts, launch announcements, public demos and public package/release history where available.	Product positioning, feature explanation, launch context and product stage.	MVP, optional
Screenshots, prototypes and demos	Uploaded screenshots and supported demo assets; store original file locally and extracted observations separately.	Visual product understanding and product showcase stories.	MVP: screenshots; video analysis later
Builder interview answers	Story-specific answers about motivation, decisions, challenges, outcomes and lessons. Attribute as builder-provided context.	Context-dependent stories and authentic first-person perspective.	MVP
Manually supplied feedback/analytics exports	Permissioned customer feedback, survey/review excerpts and CSV or manually entered verified figures.	Customer learning and milestones when supported.	MVP: manual intake

Integration roadmap

SOURCE	WHAT IT ADDS	STORY TYPES UNLOCKED	STATUS
GitHub App + selected private repositories	Read-only access to repositories explicitly selected by the builder; installation-scoped, short-lived tokens.	Private development evidence and better product understanding.	MVP; must implement and security-test
Automatic GitHub monitoring	New activity surfaced without manual scan, through webhooks or scheduled scans.	Fresh feature, bug-fix and recap opportunities.	Later phase; Coming soon
Manual PRD/document intake	Upload/paste accessible docs and notes; direct connection is not required for initial use.	Problem/solution, decisions, roadmap, product showcase.	MVP

SOURCE	WHAT IT ADDS	STORY TYPES UNLOCKED	STATUS
Public product URLs, screenshots and demos	Fetch public content or accept builder-provided material.	Showcase, feature explanation, product marketing.	MVP (supported formats only)
Direct roadmap/project-management integrations	Optional connection to planning tools after the core evidence layer proves useful.	Roadmap, experiments, audience questions.	Later phase; Coming soon
Direct product analytics integrations	Optional connection to authorized metrics sources; maintain attribution/time windows and metrics definitions.	Traction, activation, usage learning and milestones.	Later phase; manual exports first
Direct customer-feedback/support integrations	Optional connections subject to user authorization, privacy and data-minimization controls.	Customer learning, problem-solution and feature feedback.	Later phase; manual evidence first

Repository scan strategy

- Analyze a bounded recent window in the first version rather than pulling the entire repository history.
- Fetch metadata and titles first; retrieve detailed diff/comment content selectively for candidate clusters.
- Follow links between issues, PRs and commits when possible to build a coherent story evidence bundle.
- Cache only what is useful for a current scan and respect GitHub unauthenticated rate limits. Provide a retry/backoff message when rate-limited.
- Distinguish the repository's default branch state from draft branches, open PRs and unreleased work.
- For multi-contributor repositories, attribute work cautiously. Let the user optionally specify their GitHub username or focus area; do not claim another contributor's work as their own.

Product Intelligence Profile and context confirmation

During onboarding, collect the required product name and plain-language description, and ask for the current building stage. Target audience is optional initially. Allow the builder to provide a public repository URL or connect the GitHub App for selected private repositories, then optionally add PRDs, specifications, roadmap notes, changelogs, screenshots, demos, public documentation and other accessible URLs. Use available inputs to create a source-attributed Product Intelligence Profile with clear distinctions between source-supported facts, builder-provided statements and unresolved hypotheses. Let the builder review and correct the profile. When sources are unavailable or incomplete, continue with what is provided and ask adaptive follow-up questions instead of blocking onboarding.

08 / AGENT BEHAVIOR

Agent workflow, ranking and content safety

Product understanding, story discovery and drafting pipeline

STAGE	AGENT TASK	OUTPUT / RULE
1. Collect	Fetch permitted repository artifacts and any submitted documents, public URLs, screenshots and builder context.	Source records with canonical identifiers/URLs, timestamps, extraction status and access boundaries.
2. Normalize	Normalize dates, identifiers, publication/implementation state, author clues and source types.	Consistent evidence records; attribution and conflict issues flagged.

STAGE	AGENT TASK	OUTPUT / RULE
3. Build product model	Combine builder description, source observations and confirmed answers into a structured Product Intelligence Profile.	Product facts, builder statements and hypotheses stored separately; builder can correct the profile.
4. Cluster	Group related commits, PRs, issues, docs, releases and other evidence.	One candidate per coherent workstream, not one post per commit.
5. Discover angles	Use OpenAI to interpret the profile and evidence, propose categories and identify narrative possibilities.	A story hypothesis with an explainable “why it matters” and source links.
6. Verify candidate claims	Map central factual claims to support; distinguish planned vs implemented vs shipped vs measured.	Unsupported claims removed, weakened or converted into questions; no generic prompts masquerading as discovered stories.
7. Rank	Evaluate significance, evidence strength, story clarity, audience relevance, freshness and duplication.	Curated, explainable shortlist rather than opaque single score.
8. Interview builder	Create a story-specific missing-information checklist, ask one adaptive question at a time, follow up and save answers with provenance.	Enough context to tell the story, or an explicit status describing remaining gaps.
9. Build Story Brief	Set central point, audience, category, angle, event chronology, evidence links, relevant interview facts, tone and claim constraints.	Approved brief that serves as the drafting input.
10. Personalize narrative	Optionally capture the builder’s desired opening/closing style or custom wording.	Story-specific preferences, independently skippable and distinct from global voice settings.
11. Draft and verify	Generate platform-aware copy using the brief; then check claims, tone, redundancy and unsupported additions.	Editable draft with evidence drawer and flagged issues; no invented results or motivations.
12. Learn with consent	Capture corrections, edits, usefulness and optional post performance when available and authorized.	Improve relevance without overriding builder control or storing tokens/content in telemetry.

Candidate ranking and Story Readiness Check

SIGNAL	QUESTION	GUARDRAIL
Significance	Does the event contain a product change, problem, decision, lesson, meaningful plan or milestone?	Routine formatting/dependency bumps usually suppressed unless relevant to a story.
Story potential	Is there a specific problem, transformation, decision, discovery, lesson or useful update?	Do not force a narrative onto a low-signal event.
Evidence strength	Can central facts be traced to sources or clearly attributed builder answers?	A builder answer supports perspective, but does not independently prove external outcomes.
Audience relevance	Does the angle fit the known product audience and builder goal?	If unknown, favor clear progress and avoid aggressive marketing claims.
Freshness / duplication	Is this materially different from recent, drafted, published or dismissed stories?	Cluster related artifacts; do not resurface without meaningful new evidence.
Context completeness	Are necessary details present or can the builder clarify them in an interview?	Ask an adaptive sequence of questions; do not stop after one by default or interview indefinitely.

Evidence, factuality and drafting quality rules

- Every GitHub-derived candidate must include a source evidence bundle and identify which artifacts support the headline facts.
- Distinguish confirmed facts from interpretations. Example: a diff proves a code path changed, but not the founder's reason for changing it.
- Do not invent user growth, conversion, revenue, customer quotes, customer pain prevalence, emotional experiences or outcomes.
- Do not automatically post. External publishing always requires an explicit user action and a confirmation summary.
- Avoid exposing secrets, tokens, private repository details, credentials or sensitive user/customer information from source files and comments.
- When claims conflict across docs or repository state, flag the conflict or omit the claim.
- Allow dismiss, edit, correction and "already shared" feedback; do not resurface a dismissed story as new without materially new evidence.

09 / EXTERNAL ACTIONS

Buffer publishing and OAuth security

Publishing actions

ACTION	BUFFER BEHAVIOR	USER CONTROL
Save draft to Buffer	Create a post with `saveToDraft: true`; confirm success from Buffer response.	User reviews and schedules/publishes later.
Schedule	Use `createPost` with `mode: customScheduled`, a selected `channelId`, `schedulingType`, and `dueAt` in UTC.	User chooses channel, date/time/timezone and confirms.
Publish now	Use the documented `shareNow` mode and appropriate `schedulingType` for the selected channel.	Final confirmation explicitly states the content and target channel.
Queue next (future optional)	Could use Buffer queue mode where needed.	Do not add until UX/requirements call for it.

Scheduling flow

1. Choose an actual Buffer channel loaded after OAuth, not hard-coded placeholder channels.
2. Edit the post and review target-channel capabilities. Show network-specific limitations before confirmation.
3. Select publication date, time, timezone, and delivery method where supported.
4. Validate that the time is in the future; detect invalid values and handle daylight-saving ambiguity when applicable.
5. Convert the chosen local time to an ISO 8601 UTC `dueAt` timestamp for Buffer.
6. Show a final confirmation with the exact channel, date, local time/timezone and post body.
7. Submit from a server-side function; inspect the mutation result for success or typed error.
8. Only show “Scheduled” after Buffer confirms and returns a post result. On uncertain network timeouts, reconcile status before retrying to avoid duplicate posts.

OAuth/security requirements

ID	REQUIREMENT
SEC-01	Use Buffer’s OAuth 2.0 Authorization Code flow with PKCE (`S256`) as documented by Buffer.
SEC-02	Generate cryptographically random state and a code verifier per authorization attempt; validate state on callback and use an exact registered HTTPS redirect URI.
SEC-03	Keep client secret, access token, refresh token, GitHub App private key and encryption keys out of frontend code, localStorage, URLs, analytics events and logs.
SEC-04	Use server-side token exchange and Buffer/GitHub API calls. Keep confidential OAuth client secrets and GitHub App private key in Netlify environment variables.
SEC-05	Use a secure session cookie (`HttpOnly`, `Secure`, appropriate `SameSite`) and bind OAuth state/connection records to the session. Treat session identity and provider tokens separately.
SEC-06	Persist encrypted Buffer token records in Netlify Blobs; request `offline_access` where long-lived access is needed, save the latest rotating refresh token and serialize/coordinate refresh attempts. Do not rely on last-write-wins alone.
SEC-07	Request least-privilege Buffer scopes: `account:read`, `posts:write`, `offline_access`; add `posts:read` only for features that read existing posts/queues. Verify exact scopes against the current Buffer API before release.

ID	REQUIREMENT
SEC-08	Implement connect, reconnect, denied consent, expired/revoked token, disconnect, callback error and Blobs storage error states. Do not claim a grant was revoked unless Buffer confirms it.
SEC-09	Protect state-changing routes from CSRF; validate inputs; set `Cache-Control: no-store` on auth-sensitive endpoints; redact tokens from logs. Use strong consistency where immediate token-state reads are necessary.
SEC-10	Success messages must reflect provider responses. API errors must be actionable and must not expose stack traces, sensitive repository content or secrets.
SEC-11	GitHub App requests minimum read-only permissions for metadata, contents, pull requests and issues as needed by implemented endpoints. Installation tokens are short-lived and scoped to selected repositories; validate installation/repository access server-side.
SEC-12	For private-repository analysis, disclose that relevant private code/documentation may be sent to OpenAI for requested analysis/drafting; minimize excerpts, filter likely secrets, and never use private repository content in logs or telemetry.

No traditional database does not mean no secure server state

Keep product materials, source cache, Product Intelligence Profile, interviews, story candidates and drafts in browser IndexedDB; use localStorage only for small preferences. Use Netlify Blobs narrowly for encrypted persistent Buffer authorization records and minimal session-bound integration state. Netlify Blobs is a persistent key-value/object store, not a relational database. Never store OAuth tokens in browser storage. Add Export Project / Import Project backup. Browser-local projects do not automatically synchronize across devices.

10 / IMPLEMENTATION

Technical approach, local-first storage and deployment

Initial architecture proposal

COMPONENT	RESPONSIBILITY	BOUNDARY
Frontend (Next.js / React)	Marketing site, onboarding, Product Profile/source library, inbox, detail/interview, categories, draft editor and library.	No provider secrets, GitHub private key or OAuth token in browser code.
Netlify server-side functions/routes	Validate input; fetch public sources; handle GitHub App installation/token flow; orchestrate OpenAI; process Buffer OAuth callbacks, refresh and post mutations.	All sensitive actions server-side; request validation, safe errors and redacted logs.
GitHub API + GitHub App	Fetch public repository data or authorized selected-private-repository artifacts using short-lived installation tokens.	Least-privilege read-only access; repository installation scope and contributor attribution validated.
OpenAI API	Extract structured product knowledge, suggest/cluster story opportunities, produce adaptive interview questions, build Story Briefs, draft and verify copy.	Use structured evidence inputs; no unsupported claims; server-side API key; disclose processing.
IndexedDB and localStorage	IndexedDB stores Product Profile, uploaded materials, extracted text, evidence, interview answers, stories, drafts and preferences; localStorage only small UI preferences.	No OAuth tokens. Same-browser persistence only; handle quota failure and user deletion; provide export/import.

COMPONENT	RESPONSIBILITY	BOUNDARY
Buffer API + Netlify Blobs	Buffer loads authorized channels and creates draft/scheduled/immediate posts. Netlify Blobs stores encrypted Buffer token records and minimal integration state.	Server-side only; coordinate rotating refresh tokens; strong reads where needed; no token leakage.
Netlify environment variables	Store OpenAI key, GitHub App credentials, Buffer OAuth client credentials and encryption secret.	Never include values in client bundles, repository, logs or support output.

Local data model and server-side integration state

ENTITY	MINIMUM FIELDS	STORAGE
Project / ProductProfile	productId, name, description, stage, targetAudience, problem, capabilities, positioning, builderGoals, profileVersion, updatedAt	IndexedDB; each field/claim records source/provenance and confidence/status
SourceMaterial	id, sourceType, URL/fileId, title, mimeType, origin, collectedAt, eventDate, accessStatus, extractedTextRef, contentHash	IndexedDB; keep originals only where useful and user-approved; public/private source flags
EvidenceItem	id, sourceMaterialId, excerpt/observation, timestamp, artifactType, author/association, supportsClaimIds, provenanceStatus	IndexedDB; transient bounded cache for raw API artifacts
StoryCandidate	id, title, category, summary, whyItMatters, angleOptions, evidenceIds, claimMap, status, readiness, createdAt, updatedAt, dismissedAt	IndexedDB; candidates require evidence and/or clearly attributed builder context
InterviewSession / BuilderAnswer	sessionId, storyId, question, answer, answerType, answerAttribution, evidenceGap, status, reusableInProfile, consentFlag	IndexedDB; label as builder-provided; allow edit/delete and resume
StoryBrief	storyId, centralPoint, audience, purpose, narrativeAngle, chronology, evidenceIds, includedAnswers, unknowns, constraints, openingPreference, closingPreference	IndexedDB; frozen/versioned snapshot when draft generation begins
Draft	id, storyId, platform, channelId if confirmed, body, version, evidenceCheck, status, updatedAt, local/Buffer source, bufferPostId	IndexedDB; Buffer remains source of truth for Buffer post state
UserPreferences	tone preferences, content goals, timezone, category preferences, general voice observations, consent flags	IndexedDB; no sensitive inference; separate general preferences from story-specific choices
BufferAuthorization	opaque session/user key, encrypted access/refresh tokens, granted scopes, expiresAt, tokenVersion, refresh lease/lock, connection status	Netlify Blobs only; encrypted application payload; strict server-side access; no localStorage
GitHubConnection	installationId, repository IDs selected, account/owner reference, permissions snapshot, status, lastVerifiedAt	Minimal non-secret binding in secure server session/Blobs if persistence is required; generate short-lived installation token on demand

Data retention / privacy

- Fetch only repository data and source materials needed for current story discovery. Avoid cloning/copying whole repositories or retaining large raw diffs unnecessarily; use bounded caches and retain source excerpts/reference metadata.
- Clearly disclose that local drafts, source materials and profile data live in the current browser/device, do not automatically sync to other devices, and may be lost if browser data is cleared. Provide Export Project and Import Project backups.
- Provide controls to change/remove a repository and delete local Releasebuff data.
- Only ingest private repository content, customer feedback and other private material after explicit builder action/authorization. Disclose relevant AI processing, minimize data sent to OpenAI, redact likely secrets and personal data, and keep private content out of logs/telemetry.
- Publish a privacy policy before public Buffer OAuth release; describe repository access, AI processing, local storage, Buffer authorization and disconnection behavior.

Operational constraints

- Handle GitHub rate limits with caching, selective fetching, bounded scans, backoff and user-readable messages.
- Use server-side environment variables for GitHub/AI provider configuration and Buffer OAuth client secrets.
- Add request validation, timeouts, structured error codes and request IDs with redacted logs.
- Before public multi-user release, verify Netlify Blobs permissions, encryption-at-application-layer, strong reads where required, token refresh concurrency/rotation, session binding, cross-instance behavior, GitHub installation scoping, disconnect/revocation and backup/delete controls.

11 / VISUAL SYSTEM

Design direction: Tim’s Lab reference

Reference site: https://timslabs.com/ — “Tim’s Lab: Intelligent Systems for Modern Businesses.” The currently indexed homepage uses confident, outcome-led statements (“We engineer businesses for the AI Era”), compact system labels such as “SYS.01 / Diagnosis”, numbered capabilities, a clear friction/problem section, a before/after contrast, and a four-step lifecycle: Discover → Diagnose → Engineer → Deploy. This PRD adapts those structural and tonal patterns to Releasebuff rather than copying the source site’s content or assets. [1]

What to adapt

REFERENCE PATTERN	RELEASEBUFF ADAPTATION
Statement-led hero	“Build the product. Find the story worth sharing.” One promise and one clear primary CTA.
System labels / sequential modules	Use compact labels such as RBF.01 / DISCOVER, RBF.02 / INTERPRET, RBF.03 / DRAFT, RBF.04 / PUBLISH.
Friction section	Describe the manual effort: remembering what to post, reconstructing a feature story, writing from a blank page.
Numbered capability cards	Present clear features and 16 categories in structured groups, with concise descriptions.
Before / after contrast	Before: commits, PRs and notes scattered across tools. After: one evidence-backed story opportunity and an editable draft.
Explicit lifecycle	Collect signals → Find story → Review evidence → Draft → Publish.
Concrete work / project examples	Use actual source-linked story cards rather than abstract AI illustrations or unsupported metrics.

Releasebuff visual system (design proposal)

TOKEN	VALUE / RULE	USE
Primary forest	`#173D2D`	Wordmark, primary buttons, key dark panels, headings.
Lime accent	`#C9FA78`	Highlights, selected indicators, small emphasis on dark surfaces.
Warm neutral	`#F8F8F2`	Landing-page base and comfortable canvas.
Secondary panel	`#EAF0E6`	Quiet panels, grouped steps and context callouts.
Body ink	`#1E2922`	Body text with strong contrast.
Muted ink	`#5D6B61`	Supporting copy, timestamps, labels and secondary metadata.
Borders	`#D9E2D8`	Subtle dividers and card outlines.
Typography	Modern sans-serif; strong display headline; compact mono or uppercase system labels sparingly.	Short line lengths, crisp hierarchy, minimal decoration.
Shape / density	Moderately rounded cards; not excessively pill-shaped; generous overall spacing.	Professional developer-tool interface, not playful social media template.

Landing-page section order

1. Navigation: wordmark, product overview, story categories and Buffer connection entry where appropriate.
2. Hero: one strong headline, concise outcome copy, primary “Find your first story” CTA, secondary “See how it works”.
3. Product preview: realistic Story Inbox with a title, category, evidence links and draft action.
4. Problem / before-after: blank-page manual work versus ranked, evidence-backed story opportunities.
5. How it works: Describe product → Add evidence → Discover stories → Interview → Draft and refine → Publish through Buffer.
6. Story category examples grouped by purpose, not a wall of 16 large cards.
7. Show optional document, screenshot/demo and public URL intake as available; label automatic GitHub monitoring and direct third-party integrations as “Coming soon”.
8. Final CTA and restrained footer.

Workspace layout rules

- Use one stable sidebar or top-level workspace navigation for Story Inbox, Story Categories, Draft Library and Project Setup.
- The Story Inbox should be the dominant workspace screen. Cards must be scannable and evidence-forward, not tall article cards.
- Use progressive disclosure: display a concise story summary; open Story Details for long evidence lists, questions and alternate angles.
- Category labels should be visually distinct but not louder than the story title.
- Use neutral status badges; reserve strong color for meaningful states such as ready, error or selected.
- Show sample/demo data labels when using Trellis or another example project; never imply it is connected to the current user’s GitHub.

12 / UX QUALITY

Accessibility, responsive behavior and interaction states

Accessibility requirements

- Every input has a persistent label; placeholders are examples, not labels.
- Use semantic heading hierarchy, visible focus indicators and keyboard-operable controls.
- Ensure status/category meaning is conveyed by text, not color alone.
- Use readable body text and contrast ratios appropriate for text size; verify dark-panel contrast for muted copy and lime accent.
- Provide visible validation errors associated with the relevant fields and announce dynamic scan/status changes to assistive technology.
- Buttons must have specific labels (“Create draft”, “Schedule post”), not ambiguous “Continue” in every context.
- Respect reduced-motion preferences; avoid requiring animation to understand state changes.

Responsive behavior

VIEWPORT	LAYOUT RULES
Desktop / large laptop	Full workspace navigation, inbox feed with optional evidence side panel, two-column setup and a wide editor.
Tablet	Collapse navigation to compact rail/top nav; keep story card hierarchy and evidence links accessible.
Mobile	Single-column cards, compact filters, stacked editor and publishing controls, sticky primary action where useful; no horizontal text overflow.

Design states Stitch must include

SCREEN	STATES TO DESIGN
Landing	Default, primary CTA hover/focus, product preview, responsive mobile version.
Project Setup / sources	Required product name/description, idea-only path, public repo URL, GitHub App install and selected private repo, docs upload/paste, public URLs, screenshots, extraction errors, product profile review/corrections.
Story Inbox	First-time empty, analyzing, populated, filtered, category has no candidates, source unavailable, error, dismissed item.
Story Details & Interview	Evidence list, claim mapping, conflicting sources, interview gap checklist, question in progress, pause/resume/skip, answer captured, follow-up, readiness check, story brief summary.
Draft Studio	Editing, Buffer disconnected, channel selection, schedule configuration, final confirmation, request pending, successful action, API error.
Draft Library	No saved drafts, list populated, local-only draft, confirmed Buffer draft, stale or unavailable Buffer item.
Project Setup / optional sources	Manual PRD/docs/paste/URL intake and supported screenshot upload are available in MVP. Direct analytics, roadmap and feedback integrations are clearly marked “Coming soon.”
GitHub connection	Public URL mode, connect button, app install, repository selection, insufficient permission, private repository scan, revoked installation and disconnect.
Story Brief and draft quality	Angle selection, brief preview, evidence coverage, draft generated, unsupported claim flagged, user corrected claim, regeneration and ready-to-publish state.
Opening/closing preferences	Both skipped, style selected, custom wording entered, preference edited; custom wording preserved exactly in generated draft.
Backup and storage	Local save success/failure, IndexedDB quota warning, export ready, import preview/conflict, user deletes project data.

13 / VALIDATION

Success metrics and acceptance criteria

Product metrics

METRIC	DEFINITION	WHY IT MATTERS
Activation	Users who add a valid repository and complete a first successful scan / users who start setup.	Does the entry flow reach first value?
Time to first useful story	Elapsed time from valid repository entry to viewing an evidence-backed candidate.	Measures friction and discovery quality.
Story acceptance rate	Candidates opened and drafted/saved versus candidates dismissed. Track by category.	Measures whether the agent finds worthwhile stories.
Draft completion rate	Drafts created that are then saved locally, copied, or sent to Buffer.	Measures usefulness beyond generation.
Buffer connection rate	Users who authorize Buffer after initiating a Buffer action.	Identifies authorization friction.
Publishing success rate	Successful confirmed Buffer operations / valid operations attempted.	Operational reliability.
Evidence coverage	Claims in repository-derived draft that have source support / verifiable factual claims.	Protects user trust.
Duplicate suggestion rate	Repeated story candidates that add no meaningful new evidence.	Measures inbox quality.
Correction / hallucination rate	Draft claims flagged or edited because they were inaccurate or unsupported.	Quality and safety guardrail.
Product Profile correction rate	Share of profiles substantially corrected by builders after initial synthesis.	Indicates whether source discovery and inference are trustworthy.
Interview completion / readiness	Stories that reach readiness after interview, and median questions asked by category.	Measures whether interviews gather context efficiently without endless questioning.
Story Brief acceptance	Briefs accepted with little/no correction versus materially revised/rejected briefs.	Measures quality of the angle and narrative interpretation before writing.
Draft quality / edit distance	Share of drafts saved/published and amount/type of user editing, with privacy-conscious local measurement where possible.	Indicates usefulness and whether copy feels generic or inaccurate.

Analytics should be privacy-conscious. Avoid collecting post body text or OAuth tokens in telemetry. Use aggregate operational events and explicit user consent where required. Early metrics can be manually reviewed before adding an analytics integration.

Acceptance criteria — MVP

AREA	PASS CONDITION
Product onboarding	Builder can enter product name/description, choose current stage, proceed without website/repository, and optionally supply sources.
Evidence intake	Supported docs/text/public URLs/screenshots can be added; failed/inaccessible sources are reported visibly and are never silently treated as read.
Story grouping	Related artifacts form one coherent candidate; repeated scans do not flood the inbox with duplicates.
Story categories	All 16 categories are visible under four groups; category selection filters the same inbox.

AREA	PASS CONDITION
Evidence and claims	Every source-driven candidate shows source links/excerpts where available and maps central facts to evidence; builder-provided statements remain labeled separately.
Adaptive interview	AI asks a context-dependent sequence of questions, one at a time; follows up when needed; skips known facts; can pause/resume/skip; stops at readiness or records unresolved gaps.
Story Brief and drafting	Draft is generated from Product Profile + evidence + attributed interview answers + audience/angle, not only a raw commit or prompt; post-draft factuality check runs. Builder can edit, return to inbox and preserve draft/filter state.
Buffer connect	OAuth uses Authorization Code with PKCE and validated state/redirect handling; tokens never enter localStorage or logs.
Save draft	Draft creation uses Buffer's supported API operation; success is shown only after a confirmed response.
Schedule	User selects real Buffer channel, future date/time/timezone; app converts to UTC and requires final confirmation.
Publish now	User sees final content/channel confirmation; request is server-side; status only changes to published after Buffer confirms.
Error recovery	A failed or uncertain Buffer request keeps the content and provides safe next steps without accidental duplicate publishing.
Local persistence and backup	Product profile, supported source material, interviews, story candidates and drafts survive same-browser refresh where storage remains; export/import works and storage limits/errors are handled.
Responsive/accessibility	Core journey is operable by keyboard and readable on desktop, tablet and mobile; validation and status are not color-only.
Truthful capability messaging	Do not display private GitHub access or manual document intake as available until implemented and tested; automatic monitoring and later direct integrations remain "Coming soon."
GitHub App private repositories	Builder installs the GitHub App, chooses selected repos, and Releasebuff reads only granted repositories with required read-only permissions; revoked access is handled.
Buffer token persistence	Encrypted Buffer token records persist through Netlify Blobs; refresh-token rotation is serialized/locked and tested across concurrent requests/functions; no tokens in localStorage/logs.
Opening/closing personalization	Builder can skip either preference, select a style or enter custom text; custom text is preserved in the resulting draft.
Draft quality	Draft is specific to the selected product/story, natural in tone, evidence-backed, not over-templated, and does not force a CTA/question/lesson into every post.

14 / DELIVERY

Release plan, risks and open decisions

Recommended release plan

PHASE	SCOPE	EXIT CRITERIA
Phase 0 / Design baseline	Update Google Stitch prototype to reflect product description onboarding, optional sources, GitHub App, Product Profile, adaptive interview, Story Brief, optional opening/closing and Buffer actions.	Connected design covers setup → profile/evidence → inbox → detail/interview → draft → confirmed Buffer actions, with key states.

PHASE	SCOPE	EXIT CRITERIA
Phase 1 / Product understanding	Required product name/description, stage, public URL and GitHub App flow design, manual docs/paste/URL/screenshot intake, Product Intelligence Profile and local evidence model.	Builder can start without website/repo, add sources, review/correct the profile, and trace profile claims to source or builder input.
Phase 1 / Evidence-backed story discovery	Public/private repo scan, source normalization, clustering, 16 categories, claim map, dedupe, source evidence drawer and unified inbox.	Real repository artifacts create useful, deduplicated candidates; unsupported prompts are kept separate from discovered stories.
Phase 1 / Interview and draft quality	Adaptive multi-question interview, Story Readiness Check, Story Brief, category-aware narrative angles, optional opening/closing, post-draft factuality/quality review.	A candidate becomes a specific, natural draft grounded in evidence and builder input; unknown outcomes are not invented.
Phase 1 / Buffer publishing	OAuth with PKCE, Netlify Blobs encrypted token storage, secure sessions, rotation coordination, real channel listing, save draft, custom schedule, publish now, confirmations and recovery.	Test against a registered Buffer app and real test channels; no success UI without provider confirmation; concurrent refresh handled.
Phase 2 / Development automation	GitHub webhooks or scheduled monitoring; fresh evidence retrieval; candidate dedupe and feedback loop.	Meaningful new activity generates fresh, traceable opportunities without flooding the inbox.
Phase 3 / Direct integrations	Direct connections to documentation/roadmap, analytics and customer feedback sources; enhanced video/demo analysis if justified.	New sources flow through the same evidence schema with explicit permissions and claim provenance.
Phase 4 / Learning loop	Optional post performance, tone preference learning and story ranking improvements, only from available/authorized data.	Relevance improves while preserving builder control and distinguishing correlation from causality.

Key risks and mitigations

RISK	MITIGATION
Inbox noise	Cluster artifacts; suppress routine events; allow dismiss/correct; use a curated shortlist by default.
Unsupported claims / generic AI copy	Use shared evidence layer, structured Story Brief, multi-question interview where useful, provenance-aware claim check and post-draft quality review; do not invent metrics, motivations or results.
Poor product context / no website	Allow product description and idea-only onboarding; use optional docs, public URLs and visuals; editable Product Profile with unknowns clearly labeled.
Private repository exposure	GitHub App selected-repository access, read-only permissions, short-lived installation tokens, server-side checks, secret filtering, data minimization and clear AI-processing disclosure.
Buffer token exposure	Server-side OAuth, PKCE/state, secure session, encrypted token payloads in Netlify Blobs, no browser tokens, scrubbed logs.
Refresh-token race / Blobs write race	Serialize or lock refresh attempts; use strong-consistency reads when needed; do not rely on Blobs last-write-wins alone; integration-test cross-function concurrency.
Local data loss / quota	Handle IndexedDB quota/write failures; tell users data is browser-local; provide project export/import backup.
Scope creep	Keep automatic monitoring, multi-device sync and direct third-party integrations out of Phase 1; support manual intake through the shared evidence layer.
Interview fatigue	Ask only relevant questions, one at a time; show purpose/progress; support skip/pause/resume; stop at Story Readiness Check.

RISK	MITIGATION
Sensitive material in AI/logs	Minimize excerpts, redact likely secrets/personal data, never log full private source content/tokens, and disclose when materials are sent to OpenAI.

Implementation decisions to validate before release

- Validate the selected OpenAI model and structured-output schemas against the provider account/environment before release; the product provider decision is OpenAI.
- Set the bounded repository scan window and story shortlist limits; keep this configurable and documented rather than scanning entire repositories by default.
- Implement encrypted Buffer authorization records in Netlify Blobs plus concurrency-safe refresh coordination; verify strong-consistency reads and race handling in integration tests.
- Re-verify exact Buffer scopes, channels, formats and mutation schema against the live official API before release; UI must reflect the connected channel capabilities.
- The data model should support multiple local projects; confirm the interaction design can switch projects without mixing evidence or drafts.
- Define a secure application session distinct from Buffer OAuth and GitHub App installation access; do not imply provider connection is itself full product account identity.

PRD v1.1 / ADDENDUM

Product Intelligence, adaptive interviews and draft quality

This addendum records the decisions finalized after PRD v1.0. It supersedes any earlier conflicting statements in this document about public-only GitHub access, document intake, one-question context collection, token persistence and direct-from-topic drafting.

A. Product onboarding and Product Intelligence Profile

Required onboarding fields are product name and plain-language description. Ask the builder to select a current stage (exploring an idea, prototype, actively building, early launch, established product). Target audience and goals can be optional initially. Do not require a live website, repository, uploaded file or account connection to begin.

After collecting the description, offer optional source inputs: public GitHub repository URL; Connect GitHub for a GitHub App installation and selected private repositories; PRD/specification/roadmap/architecture notes/changelog via upload or paste; accessible public documentation/product/launch URLs; screenshots and supported demo assets; and manually supplied builder notes. Direct integrations with Notion, Linear, analytics and customer-support platforms are not required for MVP.

Build a reviewable Product Intelligence Profile from the sources and builder input. For each important fact, preserve provenance and distinguish (1) source-supported observation, (2) builder-provided statement, (3) inference/hypothesis awaiting confirmation, and (4) unknown. The builder can edit/correct the profile. Continue with incomplete sources rather than blocking setup, and ask only targeted questions that resolve material ambiguity.

B. Shared evidence layer and provenance

Every source adapter must emit a common SourceMaterial/EvidenceItem representation. Capture source type, canonical URL or local file identifier, source/event date where available, collection time, access status, source excerpt or observation, content hash when useful, author/attribution clues, confidentiality level and claim links. Raw materials should be minimized and stored locally unless a server-side operation requires transient processing.

Evidence labels should communicate the basis of knowledge, not model confidence alone: Source-supported, Builder-provided/confirmed, Inferred—needs confirmation, or Needs evidence. A URL existing is not sufficient evidence by itself; source relevance, excerpt and claim relationship must be visible. Distinguish planned from implemented, merged from released, and deployed from functionally verified. Customer and analytics materials require permission, minimization and careful attribution.

C. Adaptive story interview

When a builder opens a story for drafting, inspect existing sources, the Product Profile, prior approved answers and recent drafts. Create a story-specific gap checklist covering relevant dimensions: event, original problem, motivation, intended audience, approach/decision, challenge or surprise, outcome/verification, lesson/insight, next step and desired audience response. Not every story requires every dimension.

Ask one primary question at a time. Use the answer to choose the next question; follow up on vague or consequential details; skip facts already established by evidence; and stop when the story is sufficiently

complete or remaining facts can safely be omitted. Do not stop after one question by default and do not ask questions indefinitely. The builder can pause/resume, skip, edit answers and see why a question is being asked. Save answers with explicit builder-provided attribution and allow deletion. Reuse in the Product Profile only where useful and with builder control/consent.

Story Readiness Check: the central event is clear; core factual claims have suitable support; relevant perspective/context has been captured; the intended audience and angle are clear; and unresolved unknowns will not be written as facts. If an outcome is unknown, draft around the work/decision/learning instead of blocking or inventing an outcome.

D. Story Brief and draft-generation protocol

Draft generation must consume a structured Story Brief rather than a raw topic, isolated commit or generic writing prompt. The brief includes Product Profile context, event summary and chronology, evidence IDs and claim map, interview answers with attribution, audience, category, intended outcome, strongest narrative angle, relevant unknowns, voice/platform preferences, and any content restrictions. The builder can inspect or adjust the chosen angle.

The drafting pipeline is: (1) understand source-backed event, (2) propose and explain the strongest angle, (3) interview for missing context, (4) create/confirm Story Brief, (5) draft in a natural platform-aware voice, (6) run a post-draft factuality and quality review, and (7) let the builder edit, regenerate, save or publish. The post-draft review checks evidence coverage, invented claims, misattributed work, unsupported causality, product availability, metrics, repetitive wording and relevance to the chosen audience.

Quality standard: specific to this builder and product; grounded in evidence or attributed perspective; interesting because of a real problem, decision, development, insight or milestone; concise enough for the platform; natural rather than corporate/generic; and varied in opening/structure. Do not force a hook formula, inspirational lesson, question, CTA or hashtag block into every post. Factuality is checked after generation as well as before it.

E. Optional opening and closing personalization

Offer independently skippable story-specific choices for the introduction and conclusion. Opening choices may include: let AI choose the strongest opening; begin with the problem; begin with a personal thought/experience; use a specific observed moment/detail; open with a question; or supply custom text. Closing choices may include: let AI choose; end with a reflection/takeaway; ask the audience a question; invite users to explore the product; describe the next step; or supply custom text.

Never require either preference. If custom wording is supplied, preserve it faithfully and do not rewrite unnecessarily. Store these choices per story and keep them separate from general voice preferences. The selected ending must fit the story; do not force a question or CTA into every post.

F. GitHub App, Buffer and Netlify Blobs security decisions

GitHub: public URLs remain available without authorization; private repository access uses a GitHub App with minimum read-only permissions for metadata, contents, pull requests and issues required by implemented endpoints. During installation the builder selects repositories. Generate short-lived installation tokens server-side and verify that the target repository is within installation scope. Never request write access to source code for story discovery.

Buffer: use the documented OAuth 2.0 Authorization Code with PKCE flow, validated state, exact registered redirect URI, server-side token exchange and calls. Request only necessary scopes, including `posts:write`, `account:read` and `offline_access`; add `posts:read` only if reading existing posts/queues is implemented. Buffer

refresh tokens are single-use and rotate; store the latest token and coordinate refresh attempts across function instances. Real channel listing, save-to-draft, schedule and publish-now must be tested against the registered Buffer app.

Storage: keep Product Profile, source materials, evidence, interview answers, story candidates and drafts in IndexedDB; localStorage is only for small preferences. Use Netlify Blobs as persistent key-value/object storage for encrypted Buffer token records and minimal server-side integration state, protected by a server-side application-encryption key in Netlify environment variables. Use strong consistency for reads that must immediately observe token-state writes and implement a refresh lock/lease because Blobs does not provide built-in concurrency control. Do not store provider tokens in browser storage. Add Export Project and Import Project. This is a no-traditional-database architecture, not a claim that no persistence service is used.

G. Explicitly deferred capabilities

Automatic GitHub monitoring/webhooks, multi-device project sync, direct third-party integrations for documentation/roadmaps/analytics/support, advanced video understanding, shared team workspaces and centralized project storage are later-phase items unless explicitly reprioritized. Manual file, text and URL intake for supported formats is in the MVP and must not be labeled “Coming soon.”

15 / SOURCES

Reference links

The following source pages inform this PRD. Product requirements and product decisions reflect the Releasebuff planning conversation. Official provider documentation is included for implementers to verify API permissions, OAuth behavior, token rotation, persistent storage and publishing semantics before release.

Design reference

[1] Tim’s Lab — Intelligent Systems for Modern Businesses. https://timslabs.com/ Observed site patterns include outcome-led hero, SYS.01 / SYS.02 / SYS.03 labels, numbered capabilities, before/after contrast and Discover → Diagnose → Engineer → Deploy lifecycle. Accessed for this PRD on 31 October 2026.

Buffer API and OAuth

• Buffer OAuth / Authentication — https://developers.buffer.com/guides/authentication.html

• Buffer Posts & Scheduling — https://developers.buffer.com/guides/posts-and-scheduling.html

• Buffer Create Scheduled Post example — https://developers.buffer.com/examples/create-scheduled-post.html

• Buffer Create Draft Post example — https://developers.buffer.com/examples/create-draft-post.html

• Buffer CreatePostInput reference — https://developers.buffer.com/types/CreatePostInput.html

GitHub API references

• Repositories REST API — https://docs.github.com/en/rest/repos/repos

• Commits REST API — https://docs.github.com/en/rest/commits/commits

• Pull requests REST API — https://docs.github.com/en/rest/pulls/pulls

• Issues REST API — https://docs.github.com/en/rest/issues/issues

• Releases REST API — https://docs.github.com/en/rest/releases/releases

GitHub Apps and private repositories

• GitHub App installation and selected repository access — https://docs.github.com/en/apps/using-github-apps/installing-a-github-app-from-a-third-party

• GitHub installation access tokens — https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app

• GitHub App permissions — https://docs.github.com/en/apps/creating-github-apps/setting-up-a-github-app/choosing-permissions-for-a-github-app

Netlify Blobs

• Netlify Blobs — https://docs.netlify.com/build/data-and-storage/netlify-blobs/

OpenAI API

• OpenAI API documentation — https://platform.openai.com/docs/ (confirm model availability and structured-output support in the target account before implementation)

Product and interface decisions captured in v1.1

- One unified Story Inbox with all 16 categories across four groups; categories filter the same inbox. Keep unsupported prompts separate from evidence-backed discovered stories.
- Public repository URL analysis remains available without auth; private repositories are supported through a GitHub App with user-selected repositories and read-only least-privilege access. Automatic monitoring remains later phase.
- Project Setup includes MVP manual intake for product documents, notes, screenshots/demos (supported types) and accessible public URLs; direct third-party integrations for analytics, roadmaps and feedback remain later phase.
- Publishing is in scope: save to Buffer, schedule a specific time, or publish immediately, with explicit confirmation and secure OAuth.
- No traditional relational database: IndexedDB holds product materials/evidence/story/draft data; localStorage holds small preferences; Netlify Blobs holds encrypted persistent Buffer authorization state; provider secrets stay in Netlify environment variables.
- Story drafting uses a structured Story Brief, adaptive multi-question interview, post-draft evidence/quality check, and optional user-authored opening/closing to preserve the builder's natural voice.

Definition of done for the design handoff / Google Stitch should produce one coherent seven-screen experience with: product description onboarding that works without a website/repository; optional public/private GitHub connection and manual source intake; editable Product Intelligence Profile with source provenance; unified Story Inbox with 16 categories and evidence-forward cards; Story Details with adaptive interview, pause/resume/skip and readiness state; Story Brief-driven Draft Studio with optional custom opening/closing and post-draft fact checking; and real Buffer draft/schedule/publish confirmation states. Clearly distinguish MVP manual source intake from later direct integrations and automatic GitHub monitoring. Design all empty, loading, error, permission and recovery states.